



Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного
образовательного учреждения высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУК Информатика и управление

КАФЕДРА ИУК4 Программное обеспечение ЭВМ, информационные технологии

ЛАБОРАТОРНАЯ РАБОТА №1

«Введение в OpenGL»

по дисциплине: *«Компьютерная графика»*

Выполнил: студент группы ИУК4-43Б

(Подпись)

Афанасьева Ю.Р.

(И.О. Фамилия)

Проверил:

(Подпись)

Тихонович Е.О.

(И.О. Фамилия)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:

- Оценка:

Цель: формирование практических навыков по работе с проекционной матрицей средствами OpenGL, а также созданию простейших анимаций графических примитивов и их адаптации в абсолютных и относительных оконных координатах.

Задачи:

сформировать представление о методах и секторе решаемых OpenGL задач, изучить основные принципы работы OpenGL, представлять и понимать основные реализации OpenGL, знать типы данных OpenGL и специфику именования переменных, понимать основные принципы трехмерного программирования компьютерной графики, иметь представление о проекциях, уметь создавать типовый проект в различных средах разработки (Visual Studio), иметь представление о двойной буферизации.

Задание выполняется согласно варианту. По завершении готовится отчет.

- 1) Для [Листинга 1](#) знать все функции и уметь объяснить основные входные параметры
- 2) Для [Листинга 2](#) поменять цвет фона и название окна
- 3) Для [Листинга 3](#) и [Листинга 4](#) поменять цвет фона, цвет объекта, координаты объекта и количество объектов. Очень важно понимать работу функций `void glViewport(GLint x, GLint y, GLsizei width, GLsizei height)` и `void glOrtho(GLdouble left, GLdouble right, GLdouble bottom, GLdouble top, GLdouble near, GLdouble far)`. Продемонстрировать работу программы с не менее чем 5 разными вариантами параметров для каждой из этих функций. Объяснить влияние каждой функции на вывод графических объектов.
- 4) Для [Листинга 5](#) реализовать следующие изменения согласно варианту.

Вариант 3

3) выполнить вращение прямоугольника

Листинг 2

```
#include "glut.h"

void RenderScene(void)
{
    // Окно очищается текущим цветом очистки
    glClear(GL_COLOR_BUFFER_BIT);
    // В буфер вводятся команды рисования
    glFlush();
}

// Устанавливается состояние визуализации
void SetupRC(void)
```

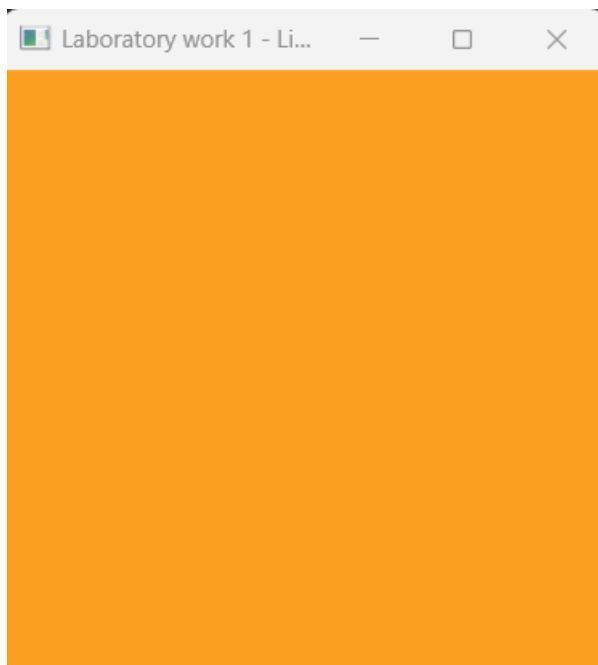
```

{
    glClearColor(0.98f, 0.625f, 0.12f, 1.0f);
}

int main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutCreateWindow("Listing 2");
    glutDisplayFunc(RenderScene);
    SetupRC();
    glutMainLoop();
    return 0;
}

```

Результаты 2



Листинг 3 и 4

```

#include "glut.h"
#include <stdio.h>

int currentVariant = 1;
GLfloat bgColor[4] = { 0.0f, 0.0f, 1.0f, 1.0f };
GLfloat objColor[3] = { 1.0f, 0.0f, 0.0f };
GLfloat rectCoords[4] = { -25.0f, 25.0f, 25.0f, -25.0f };
int numObjects = 1;
GLsizei windowWidth = 800;
GLsizei windowHeight = 600;

void RenderScene(void)
{

```

```

glClear(GL_COLOR_BUFFER_BIT);
for (int i = 0; i < numObjects; i++)
{
    GLfloat offset = i * 30.0f;
    glColor3f(objColor[0], objColor[1], objColor[2]);
    glRectf(rectCoords[0] + offset, rectCoords[1] + offset,
            rectCoords[2] + offset, rectCoords[3] + offset);
}
glFlush();
}

void SetupRC(void)
{
    glClearColor(bgColor[0], bgColor[1], bgColor[2], bgColor[3]);
}

void ChangeSize(GLsizei w, GLsizei h)
{
    GLfloat aspectRatio;
    if (h == 0) h = 1;
    windowWidth = w;
    windowHeight = h;

    if (currentVariant == 1)
    {
        glViewport(0, 0, w, h);
        glMatrixMode(GL_PROJECTION);
        glLoadIdentity();
        aspectRatio = (GLfloat)w / (GLfloat)h;
        if (w <= h)
            glOrtho(-100.0, 100.0, -100.0 / aspectRatio, 100.0 / aspectRatio,
1.0, -1.0);
        else
            glOrtho(-100.0 * aspectRatio, 100.0 * aspectRatio, -100.0, 100.0,
1.0, -1.0);
    }
    else if (currentVariant == 2)
    {
        glViewport(w / 2, 0, w / 2, h);
        glMatrixMode(GL_PROJECTION);
        glLoadIdentity();
        glOrtho(-100.0, 100.0, -100.0, 100.0, 1.0, -1.0);
    }
    else if (currentVariant == 3)
    {
        glViewport(0, 0, w / 2, h);
        glMatrixMode(GL_PROJECTION);
        glLoadIdentity();
        glOrtho(-50.0, 50.0, -50.0, 50.0, 1.0, -1.0);
    }
    else if (currentVariant == 4)
    {

```

```

        glViewport(w / 4, h / 4, w / 2, h / 2);
        glMatrixMode(GL_PROJECTION);
        glLoadIdentity();
        glOrtho(-200.0, 200.0, -200.0, 200.0, 1.0, -1.0);
    }
    else
    {
        glViewport(w / 3, 0, w / 3, h);
        glMatrixMode(GL_PROJECTION);
        glLoadIdentity();
        glOrtho(-150.0, 150.0, -150.0, 150.0, 1.0, -1.0);
    }
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

void Keyboard(unsigned char key, int x, int y)
{
    switch (key)
    {
        case '1':
            currentVariant = 1;
            bgColor[0] = 0.0f; bgColor[1] = 0.0f; bgColor[2] = 1.0f;
            objColor[0] = 1.0f; objColor[1] = 0.0f; objColor[2] = 0.0f;
            rectCoords[0] = -25.0f; rectCoords[1] = 25.0f; rectCoords[2] = 25.0f;
rectCoords[3] = -25.0f;
            numObjects = 1;
            break;
        case '2':
            currentVariant = 2;
            bgColor[0] = 1.0f; bgColor[1] = 0.4f; bgColor[2] = 0.7f;
            objColor[0] = 1.0f; objColor[1] = 1.0f; objColor[2] = 0.0f;
            rectCoords[0] = -50.0f; rectCoords[1] = 50.0f; rectCoords[2] = 50.0f;
rectCoords[3] = -50.0f;
            numObjects = 2;
            break;
        case '3':
            currentVariant = 3;
            bgColor[0] = 1.0f; bgColor[1] = 0.0f; bgColor[2] = 0.0f;
            objColor[0] = 1.0f; objColor[1] = 1.0f; objColor[2] = 0.0f;
            rectCoords[0] = -75.0f; rectCoords[1] = 75.0f; rectCoords[2] = 75.0f;
rectCoords[3] = -75.0f;
            numObjects = 3;
            break;
        case '4':
            currentVariant = 4;
            bgColor[0] = 1.0f; bgColor[1] = 1.0f; bgColor[2] = 0.0f;
            objColor[0] = 1.0f; objColor[1] = 0.5f; objColor[2] = 0.0f;
            rectCoords[0] = -30.0f; rectCoords[1] = 30.0f; rectCoords[2] = 30.0f;
rectCoords[3] = -30.0f;
            numObjects = 4;
    }
}

```

```

        break;
    case '5':
        currentVariant = 5;
        bgColor[0] = 1.0f; bgColor[1] = 0.5f; bgColor[2] = 0.0f;
        objColor[0] = 0.0f; objColor[1] = 0.75f; objColor[2] = 1.0f;
        rectCoords[0] = -20.0f; rectCoords[1] = 20.0f; rectCoords[2] = 20.0f;
rectCoords[3] = -20.0f;
        numObjects = 5;
        break;
    default:
        return;
    }
    SetupRC();
    ChangeSize(windowWidth, windowHeight);
    glutPostRedisplay();
}

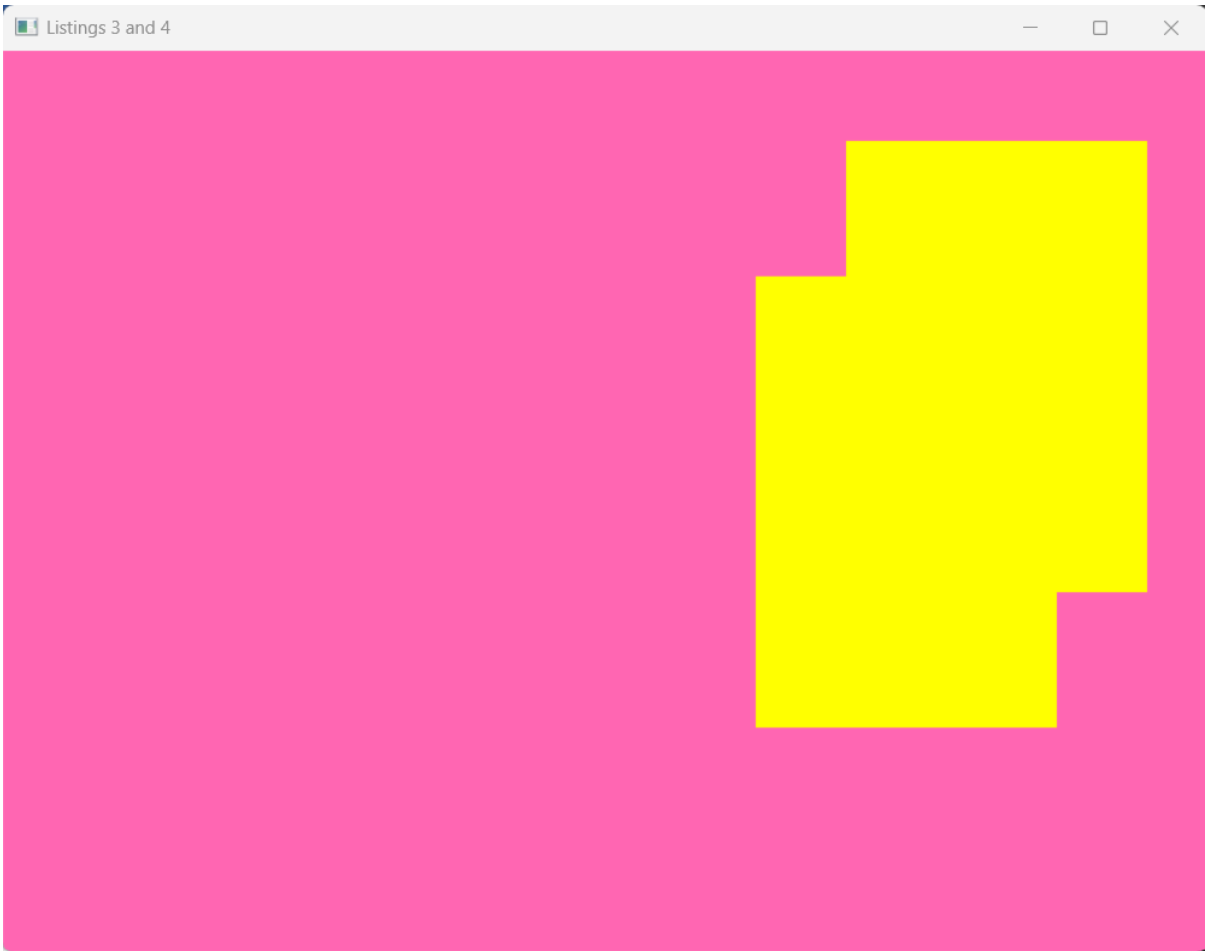
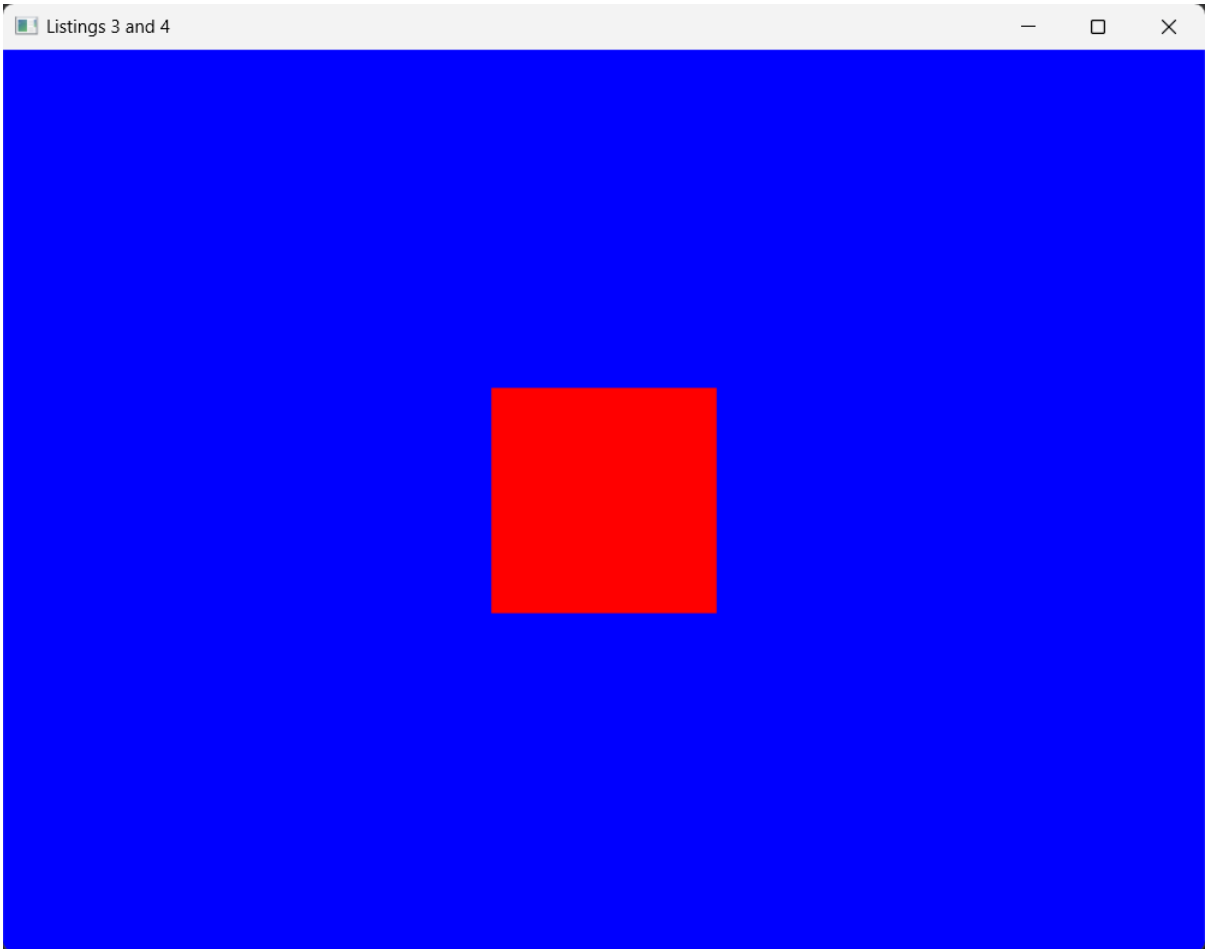
int main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(800, 600);
    glutCreateWindow("Listings 3 and 4");
    glutDisplayFunc(RenderScene);
    glutReshapeFunc(ChangeSize);
    glutKeyboardFunc(Keyboard);
    SetupRC();
    glutMainLoop();
    return 0;
}

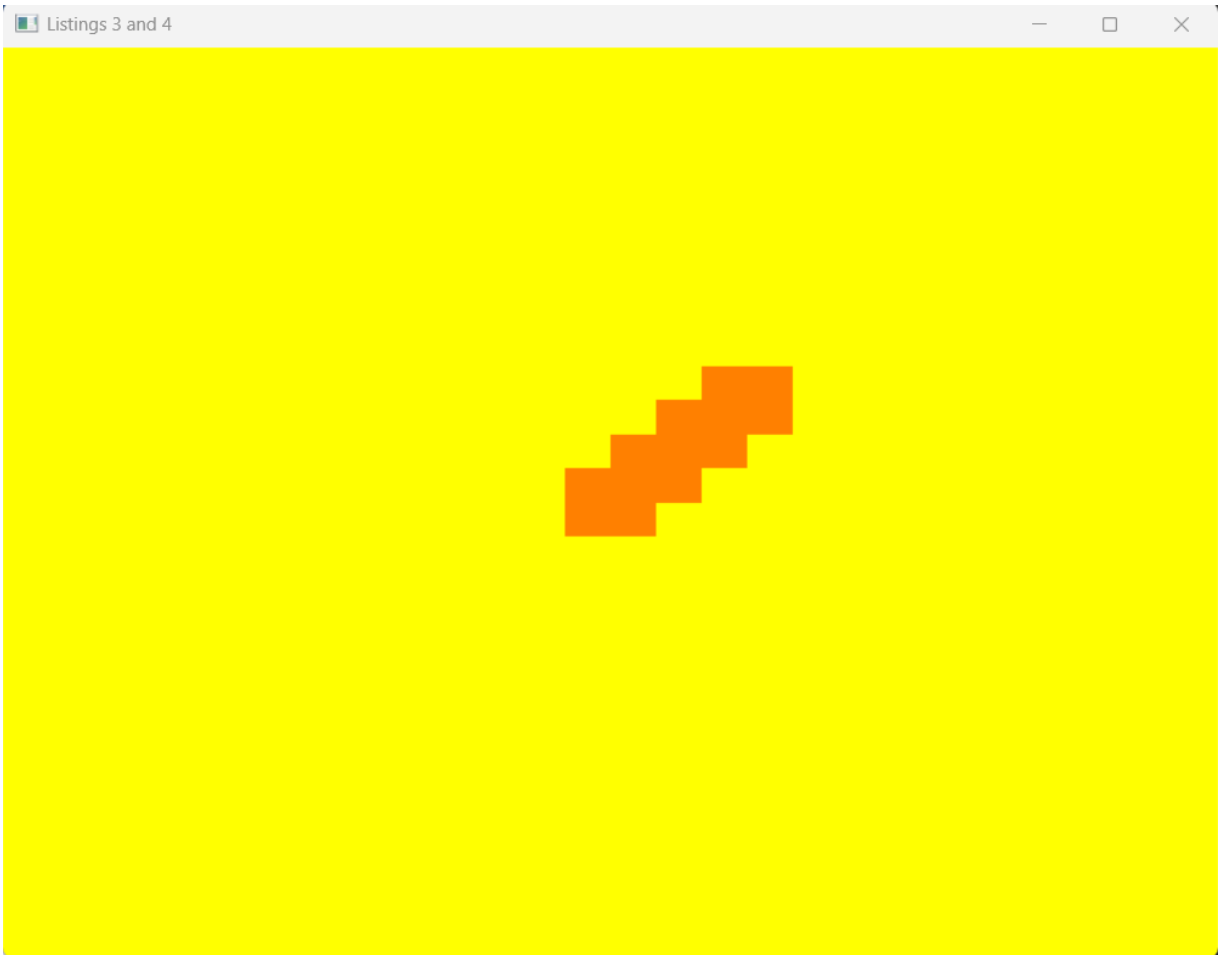
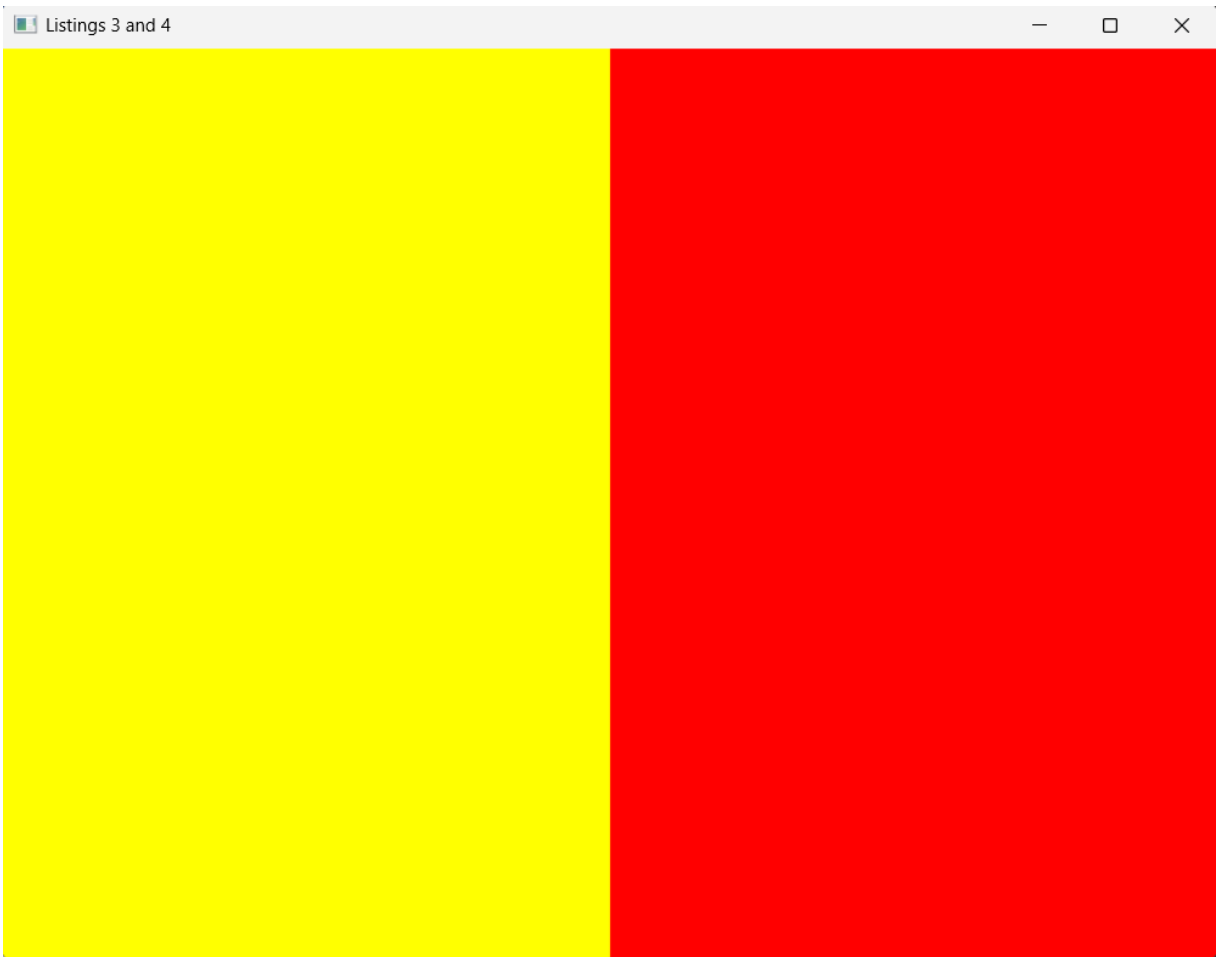
```

Результаты 3 и 4

`glViewport(x, y, width, height)` задаёт прямоугольную область окна, в которую выводится изображение. Параметры `x` и `y` определяют положение её левого нижнего угла, а `width` и `height` — размеры. Изменение положения перемещает область вывода, а изменение размеров масштабирует изображение и может искажать его пропорции.

`glOrtho(left, right, bottom, top, near, far)` задаёт границы видимого ортогографического объёма. Уменьшение диапазонов по `X` и `Y` визуально увеличивает объекты, увеличение диапазонов — уменьшает. Объекты за указанными границами отсекаются. Параметры `near` и `far` задают границы отсечения по оси `Z`.





Листинг 5

```
#include "glut.h"

GLfloat angle = 0.0f;

GLfloat windowHeight;
GLfloat windowHeight;

void RenderScene(void)
{
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0f, 0.0f, 0.0f);
    glPushMatrix();
    glRotatef(angle, 0.0f, 0.0f, 1.0f);
    glRectf(-40.0f, 20.0f, 40.0f, -20.0f);
    glPopMatrix();
    glutSwapBuffers();
}

void TimerFunction(int value)
{
    angle += 1.0f;
    if (angle > 360.0f)
        angle -= 360.0f;
    glutPostRedisplay();
    glutTimerFunc(33, TimerFunction, 1);
}

void SetupRC(void)
{
    glClearColor(0.0f, 0.0f, 1.0f, 1.0f);
}

void ChangeSize(GLsizei w, GLsizei h)
{
    GLfloat aspectRatio;
    if (h == 0)
        h = 1;

    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    aspectRatio = (GLfloat)w / (GLfloat)h;
    if (w <= h)
    {
        windowHeight = 100;
        windowHeight = 100 / aspectRatio;
        glOrtho(-100.0, 100.0, -windowHeight, windowHeight, 1.0, -1.0);
    }
    else
```

```

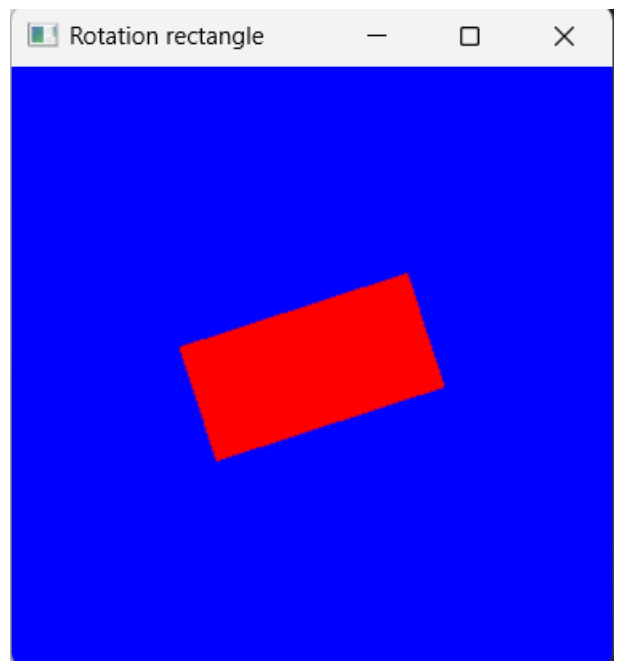
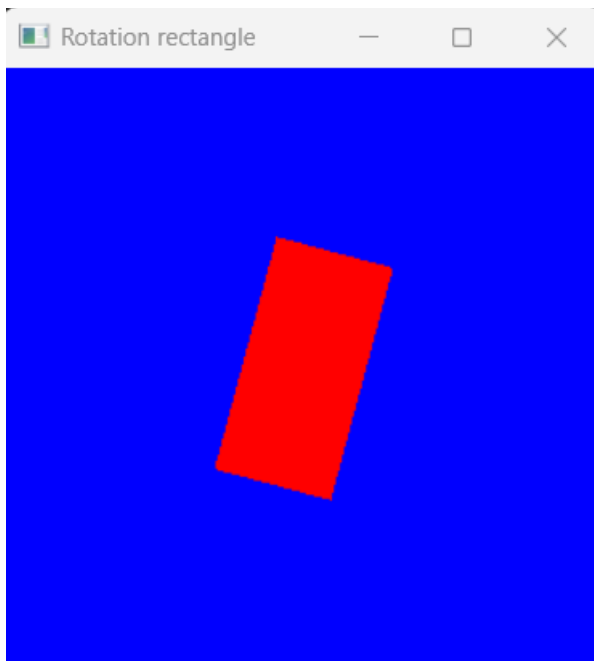
{
    windowHeight = 100 * aspectRatio;
    windowHeight = 100;
    glOrtho(-windowWidth, windowWidth, -100.0, 100.0, 1.0, -1.0);
}

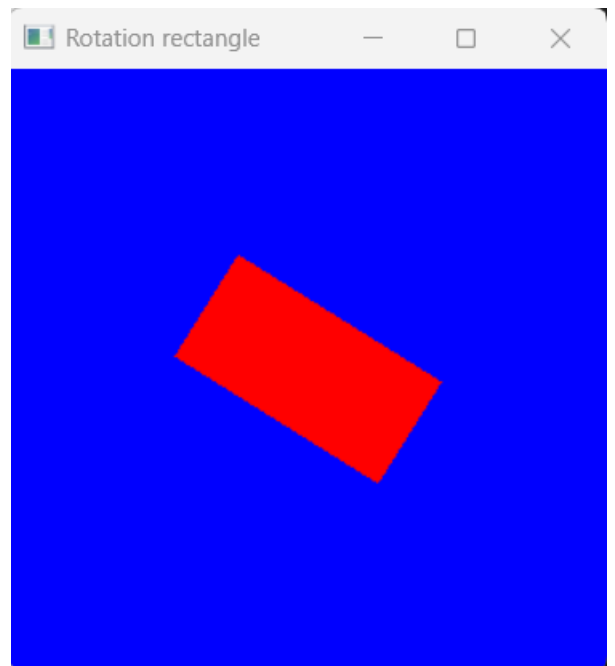
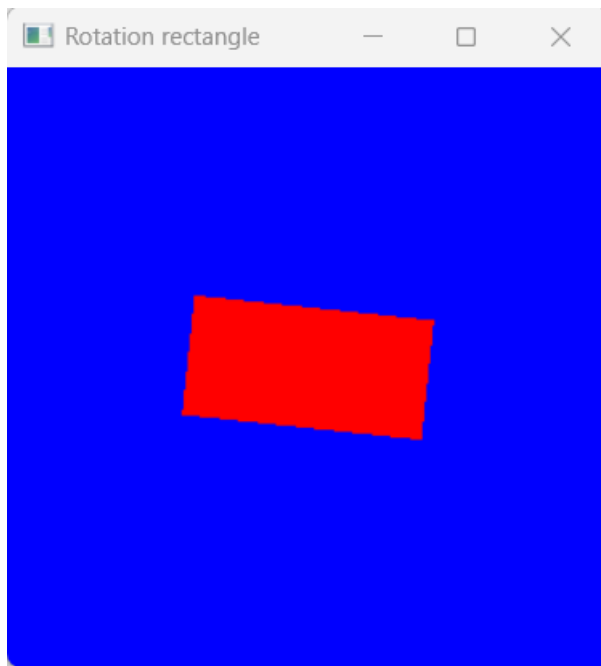
glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
}

int main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutCreateWindow("Rotation rectangle");
    glutDisplayFunc(RenderScene);
    glutReshapeFunc(ChangeSize);
    glutTimerFunc(33, TimerFunction, 1);
    SetupRC();
    glutMainLoop();
    return 0;
}

```

Результаты 5





Вывод: в ходе работы были получены практические навыки по работе с проекционной матрицей средствами OpenGL, а также созданию простейших анимаций графических примитивов и их адаптации в абсолютных и относительных оконных координатах.